

A Machine Learning-Based Security Monitoring and Threat Detection Platform

Document Type: Project Proposal & Software Requirements Specification (SRS)

Prepared By:

Shreshth Garg (1024030380)
Geetansh Bansal (1024030954)
Lakshit Goyal (1024030383)

Team Name:

HackViz

Submitted to:

Dr. Stuti Chug



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Section No.	Section Title	Page No.
—	FRONT PAGE / COVER	i
—	TABLE OF CONTENTS	ii
1.0	PROJECT SELECTION PHASE	1
1.1	Problem Statement & Proposed Solution	1
1.2	Scope & Evaluation Criteria	2
2.0	PROJECT OVERVIEW	3
2.1	CERT Dataset & System Architecture	3
2.2	Machine-Learning Models & Overall Workflow	4
3.0	ANALYSIS PHASE	5
3.1	Use Cases & Use Case Templates	5
3.2	Activity, Swimlane, & Data Flow Diagrams (DFDs)	8
4.0	SOFTWARE REQUIREMENT SPECIFICATION (IEEE FORMAT)	12
4.1	Product Perspective & Functional Requirements	12
4.2	Non-Functional & Data Requirements	14
4.3	ML Requirements & User Stories	15
5.0	DEVELOPMENT PROGRESS AND WEEKLY JOURNAL	17
5.1	Weekly Milestones (Weeks 1 to 3)	17
6.0	PROJECT STATUS & FUTURE DIRECTIONS	19
6.1	Current Progress, Next Steps, & Limitations	19

1.PROJECT SELECTION PHASE

1.1 Software Bid

Higher-Order Goal:

The goal of the project is to develop a behavior-based security monitoring system that helps security analysts identify unusual user or system activity. The system aims to reduce dependence on rigid rules and predefined thresholds while providing meaningful anomaly rankings and supporting information for analyst investigation. Ed

Problem Statement:

Traditional SIEM systems commonly depend on static rules and predefined thresholds, such as detecting a fixed number of failed login attempts. Although these rules are useful for known patterns, they can generate a large number of false positives from legitimate employee behavior and may fail to detect sophisticated or previously unseen attacks that do not match any predefined rule, creating a need for a system that can learn normal behavioral patterns and identify deviations without requiring manually written rules for every possible attack.

Proposed Solution:

The model learns to reconstruct normal sequences. When an activity sequence is significantly different from the learned behavior, its reconstruction error becomes higher and it is assigned a higher anomaly score. Suspicious sessions are then presented to a human analyst along with relevant behavioral information and explanations.

Solution Approach:

To determine which architecture is most suitable for behavioral anomaly detection, three model variants will be compared using a common preprocessing and evaluation pipeline:

- **LSTM/GRU Autoencoder:** Used as the baseline sequential model for learning local and short-range dependencies between events.
- **Transformer Autoencoder:** Uses self-attention to investigate whether long-range relationships between events can be modelled more effectively.
- **Hybrid LSTM + Attention:** Combines the order-sensitive nature of recurrent networks with attention-based modelling of important events and longer dependencies.

This comparison will help determine the trade-offs between detection performance, computational cost and explainability.

Evaluation Criteria:

The primary evaluation metric will be F1-score, as anomaly-detection datasets generally contain a severe imbalance between normal and malicious activities. The following secondary measures will also be considered:

- False-positive rate with different amounts of available behavioral history.
- Training time required by each model.
- Inference latency during anomaly scoring.
- Usefulness of generated explanations.
- Analyst-perceived clarity of suspicious activity.

These measures will allow the project to evaluate not only whether a model detects anomalies, but also whether its results are practical for security analysts.

Time-to-Value and Scalability:

The system is intended to provide useful anomaly rankings from learned behavioral patterns without requiring manually authored rules for every possible attack. Transfer learning will be investigated to determine whether a model trained on existing behavioral patterns can be adapted to new entities with less entity-specific historical data.

The proposed architecture separates the Java/Spring Boot application layer from the Python ML service. This separation allows the machine-learning component to evolve independently and provides the possibility of deploying and scaling the ML service separately in future versions.

Scope

The project covers the complete workflow required for behavioral anomaly analysis:

- Behavioral log ingestion.
- Data preprocessing and normalization.
- Sessionization and sequence generation.
- Anomaly detection and scoring.
- Generation of supporting explanations.
- Analyst review of suspicious sessions.
- Confirm/dismiss feedback collection.
- Transfer-learning experiments.

Automatic blocking, account suspension or other autonomous enforcement actions are intentionally outside the project scope. The system is designed as an analyst-assistance platform.

Risks and Mitigation:

The major risks include class imbalance, limitations of synthetic or benchmark data, preprocessing complexity, cold-start behaviour and false positives. These risks will be addressed through suitable evaluation metrics, clear documentation of dataset limitations, efficient preprocessing, transfer-learning experiments and a human-in-the-loop review process.

Summary:

The software bid proposes an experimental security-monitoring platform that learns behavioural patterns and identifies suspicious deviations. The focus is on assisting analysts with anomaly rankings and explanations rather than replacing human decision-making or automatically enforcing security actions.

1.2 Project Overview

The Behavioral Anomalies system is designed to detect unusual user and system activity by learning normal behavioural sequences. Instead of examining individual events in isolation, the system considers the order, relationship and timing of activities within a session.

For example, a normal user may typically follow a sequence such as login → dashboard access → file access → logout. If a session contains an unusual combination or sequence of activities, the system can assign a higher anomaly score and bring it to the analyst's attention.

Dataset:

The primary evaluation dataset for the project is the CERT Insider Threat Test Dataset. It contains multiple sources of activity data, including:

- Logon records
- HTTP activity
- File activity
- Device usage
- Email records

The dataset also provides ground-truth malicious scenarios, which can be used to evaluate how effectively the proposed models identify anomalous behaviour.

Psychometric information is deliberately excluded from the model features. The system therefore focuses on observable digital activity sequences rather than personal psychological characteristics.

System Architecture:

1. Spring Boot Backend:

The backend forms the application layer of the system. It is responsible for log ingestion, session windowing, role-based access control, audit logging and communication with the Python machine-learning service.

2. Python ML Service:

The ML service handles the machine-learning workflow, including preprocessing, sequence generation, model training, inference and anomaly scoring. It will also support attention extraction and transfer-learning experiments required for comparing the proposed architectures.

3. Streamlit Analyst Dashboard:

The dashboard provides the interface for security analysts. It will allow analysts to review suspicious sessions, inspect behavioural timelines, view anomaly explanations and provide confirm/dismiss feedback on detected anomalies.

Machine-Learning Models:

Model	Role in the Project
LSTM/GRU Autoencoder	Baseline sequential model for learning local and short-range behavioural dependencies
Transformer Autoencoder	Evaluates whether self-attention can improve modelling of long-range relationships between events.
Hybrid LSTM + Attention	Combines the order-sensitive representation of recurrent layers with attention-based long-range modelling.

All three models will use a common preprocessing and evaluation pipeline so that their performance can be compared fairly.

Overall Workflow:

Behavioural Logs → Preprocessing → Sessionization → Sequence Generation → ML Model → Anomaly Score → Explanation → Analyst Review → Feedback

This workflow allows the system to move from raw behavioural records to analyst-ready suspicious-session information. The analyst remains responsible for reviewing the evidence and deciding whether an activity is genuinely suspicious.

Expected Outcome:

The expected outcome is a prototype platform capable of identifying behavioural anomalies from heterogeneous activity logs and presenting them in a clear form for security-analyst investigation. The comparison of LSTM/GRU, Transformer and hybrid architectures will provide insight into which approach offers the best balance of detection accuracy, false-positive control, computational efficiency and explainability.

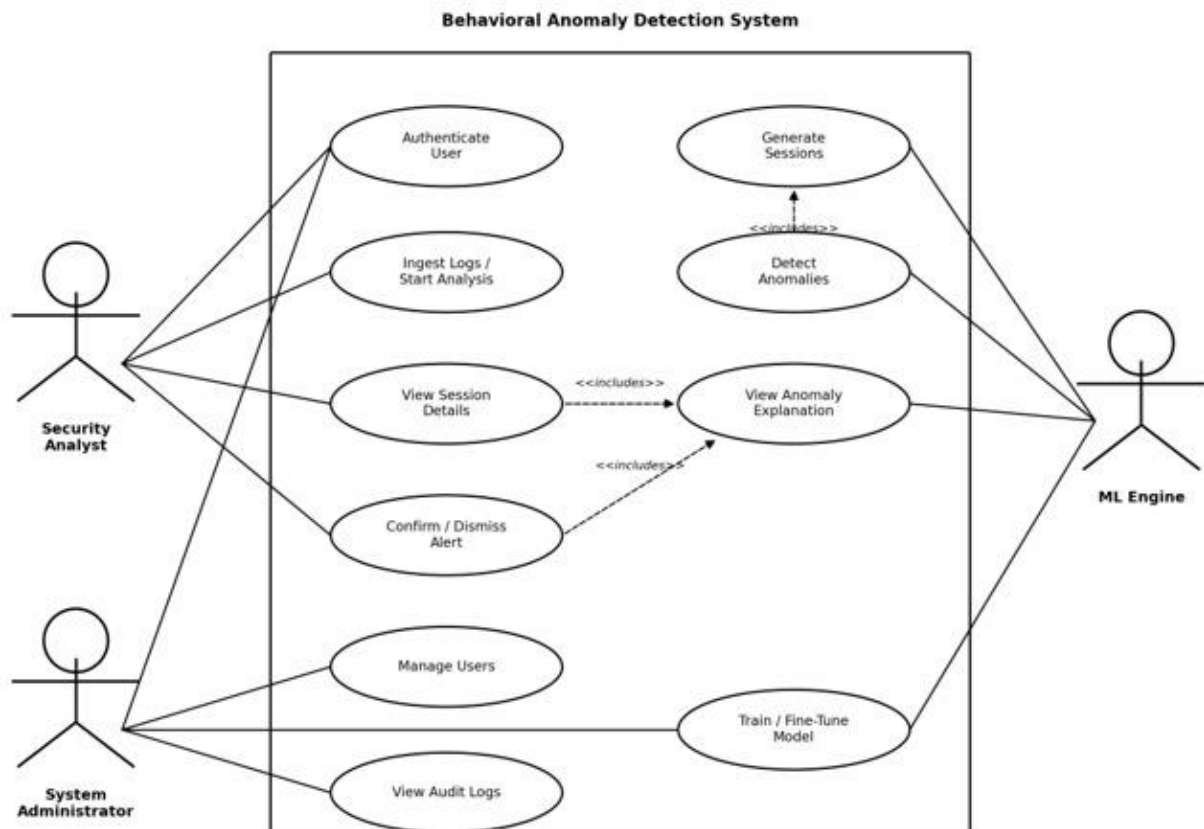
Overall, the project aims to demonstrate how behaviour-based machine learning can complement traditional security monitoring by identifying deviations that may not be captured by fixed, manually defined rules.

2. ANALYSIS PHASE

2.1 Use Cases:

The system supports two human actor types — the Security Analyst and the System Administrator — while the ML Engine is modelled as a system-level actor that participates in anomaly analysis. The use-case diagram below captures the major interactions between these actors and the system.

2.1.1 Use-Case Diagram:



2.1.2 Use Case Templates:

Use Case ID	UC-01
Use Case Name	Authenticate User
Primary Actor	Security Analyst / Administrator
Precondition	User account exists and the authentication service is available.
Main Flow	User submits credentials; the system validates the credentials and role; access is granted or denied.
Postcondition	An authenticated session is created for valid users.

Use Case ID	UC-02
Use Case Name	Ingest Logs
Primary Actor	Security Analyst / System
Precondition	A supported log source or file is available.
Main Flow	The system receives logs, validates the format, stores raw/normalised records and reports ingestion status.
Postcondition	Logs are made available for preprocessing.

Use Case ID	UC-03
Use Case Name	Generate Sessions
Primary Actor	ML Engine
Precondition	Normalised events with valid timestamps and entity identifiers exist.
Main Flow	Events are ordered and grouped according to the configured sessionization strategy.
Postcondition	Behavioural sequences are generated.

Use Case ID	UC-04
Use Case Name	Detect Anomalies
Primary Actor	ML Engine
Precondition	A trained model and a valid sequence are available.
Main Flow	The sequence is encoded; the model reconstructs it; the reconstruction error is calculated and a threshold is applied.
Postcondition	The session receives an anomaly score / classification.

Use Case ID	UC-05
Use Case Name	View Session Details
Primary Actor	Security Analyst
Precondition	Suspicious or historical sessions exist.
Main Flow	The analyst selects a session and views the chronological events, score and relevant metadata.
Postcondition	Session details are displayed to the analyst.

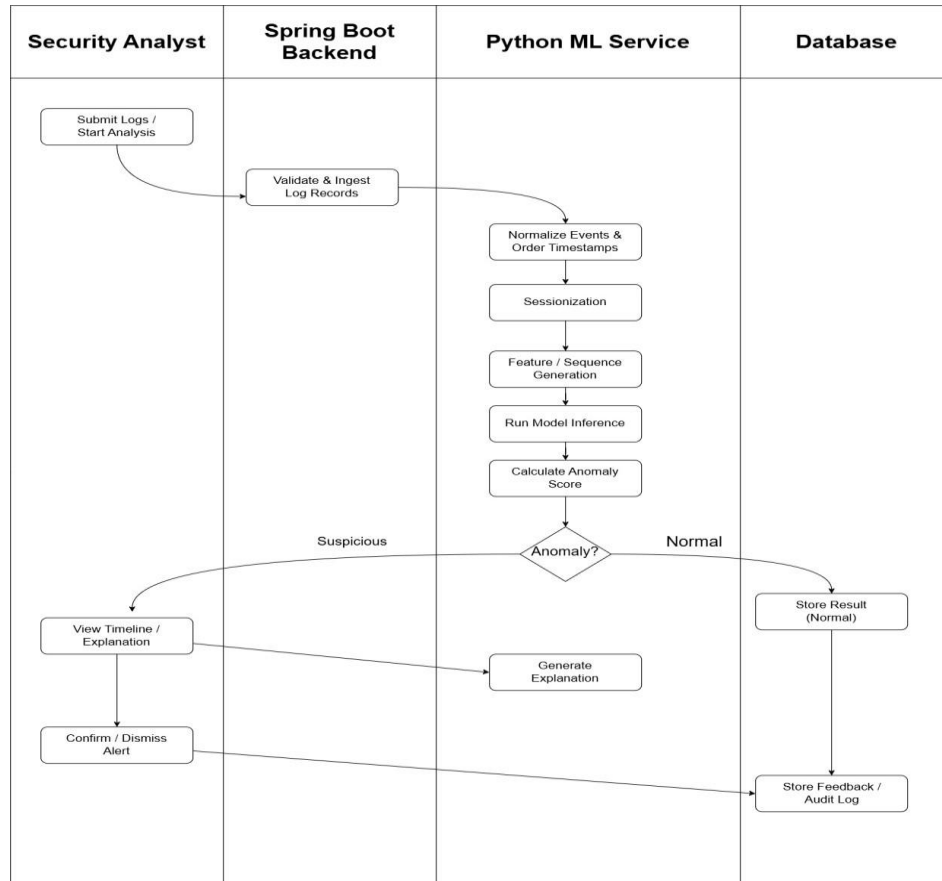
Use Case ID	UC-06
Use Case Name	View Anomaly Explanation
Primary Actor	Security Analyst
Precondition	An anomaly has already been scored by an explainable model.
Main Flow	The system presents the contributing sequence regions/events and attention information where available.
Postcondition	The analyst receives supporting evidence for triage.

Use Case ID	UC-07
Use Case Name	Confirm / Dismiss Alert
Primary Actor	Security Analyst
Precondition	An alert is displayed to the analyst.
Main Flow	The analyst reviews the evidence and marks the alert as confirmed or dismissed.
Postcondition	Feedback is recorded for audit and future experimentation.

Use Case ID	UC-08
Use Case Name	Train / Fine-Tune Model
Primary Actor	Administrator / ML Engineer
Precondition	Training data and configuration are available.
Main Flow	The system trains a base model or fine-tunes it using local/entity-specific data.
Postcondition	An updated model is saved for inference.

2.2 Activity Diagram and Swimlane Diagrams:

The principal activity flow begins with log ingestion and ends with analyst review. The flow is designed to separate data preparation from model inference and human decision-making, and is organised into swimlanes by responsible component.



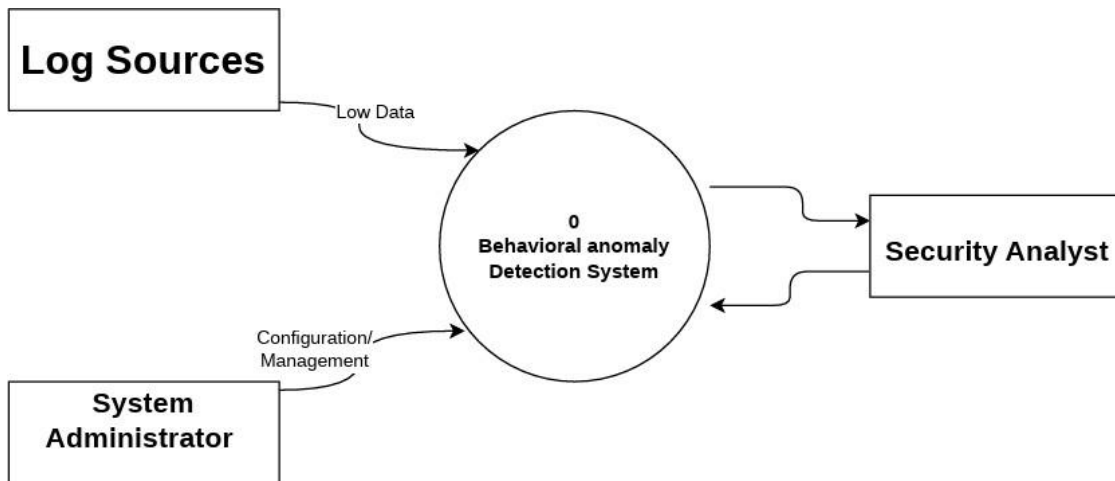
Swimlane Responsibility Summary:

Security Analyst	Spring Boot Backend	Python ML Service	Database
Authenticate	Validate request	-	Read user record
Start analysis	Receive logs	-	Store raw records
-	Send session data	Preprocess + sessionize	Store processed data
-	Request inference	Run model + calculate score	Read model / config
Review alert	Return results	Generate explanation	Store alert
Confirm / dismiss	Record feedback	Optional future retraining input	Store audit / feedback

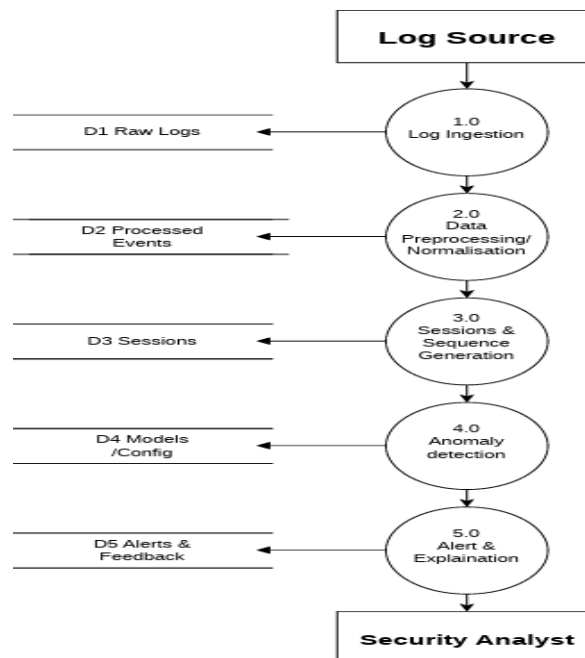
2.3 Data Flow Diagrams (DFDs):

The DFDs move from a high-level system view (Level 0) down to the internal anomaly-detection process (Level 2).

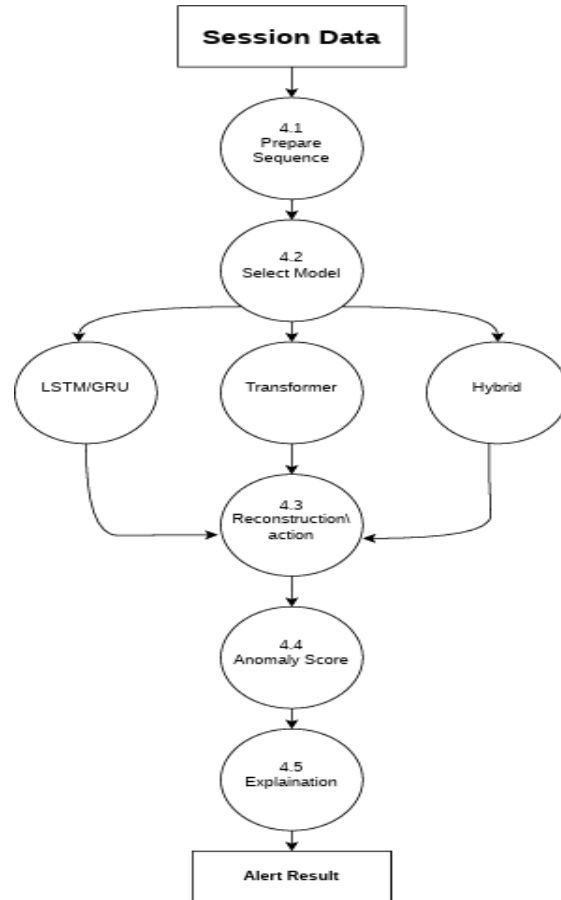
2.3.1 DFD Level 0 — Context Diagram:



2.3.2 DFD Level 1:



2.3.3 DFD Level 2 — Decomposition of Process 4.0 (Anomaly Detection):



Data Stores:

- D1 — Raw Logs
- D2 — Processed Events
- D3 — Sessions
- D4 — Model / Config
- D5 — Alerts & Feedback

2.4 Software Requirement Specification in IEEE Format

2.4.1 Introduction

Purpose: This Software Requirements Specification describes the functional and non-functional requirements of the Behavioral Anomaly Detection System.

Scope: The system will take behavioural logs, group them into user/entity sessions, detect unusual activity using deep-learning autoencoders, and show the results to security analysts.

Definitions: SIEM — Security Information and Event Management; UEBA — User and Entity Behavior Analytics; Anomaly — activity that is very different from the learned normal behaviour; Session — a group of related events arranged within a certain time period.

Overview: The system consists of three main parts: a Java/Spring Boot backend, a Python ML service, and a Streamlit interface for security analysts.

2.4.2 Overall Description

Product Perspective: The user-facing application and the machine-learning service will be kept separate. This makes it easier to test and change different ML models without changing the complete application.

Product Functions: The main functions of the system include user login, log collection, data checking, data preparation, session creation, anomaly detection, alert ranking, explanation of results, analyst feedback, model training/fine-tuning and activity logging.

User Classes: The system will have three main types of users:

- Security Analyst
- System Administrator
- ML/Model Administrator

Operating Environment: The system will use a Spring Boot backend running on Java, a Python-based ML service, a Streamlit interface and a database for storing required data.

Constraints: The system must work with highly unbalanced security data. The main dataset is synthetic, inference should be fast enough for analyst review, and model explanations should only be treated as supporting information and not as final proof of malicious activity.

Assumptions: The input logs are expected to contain useful timestamps and user/entity IDs. Only authorised users will have access to the required logs, and trained model files will be available before the system is used for detection.

2.4.3 Functional Requirements:

ID	Requirement
FR-01	The system shall authenticate users and enforce role-based access to protected functionality.
FR-02	The system shall ingest supported behavioural log files or records.

ID	Requirement
FR-03	The system shall validate and normalise heterogeneous event records into a common representation.
FR-04	The system shall order events using timestamps and generate behavioural sessions.
FR-05	The system shall transform sessions into model-compatible sequences / features.
FR-06	The system shall calculate an anomaly score for each analysed session.
FR-07	The system shall rank or filter sessions for analyst triage using configurable criteria.
FR-08	The system shall display chronological session details and relevant metadata.
FR-09	The system shall display available explainability information, including attention-based evidence for supported models.
FR-10	The system shall allow an analyst to confirm or dismiss an alert.
FR-11	The system shall record analyst feedback and audit events.
FR-12	The system shall support model training and entity-specific fine-tuning for experiments.

2.4.4 Non-Functional Requirements

Category	Requirement
Performance	The system should provide inference latency suitable for interactive analyst triage and should record that latency.
Security	Authentication, authorisation and audit logging shall protect sensitive security-monitoring operations.
Reliability	Invalid or incomplete log records should be handled without corrupting the remaining valid data.
Scalability	The ML service and backend should remain separable so either component can be scaled independently.
Usability	Analysts should be able to identify suspicious sessions, inspect timelines and understand the available explanations.
Maintainability	Preprocessing, model training, inference and UI components should remain modular.
Explainability	Model-generated evidence shall be presented as supporting information rather than definitive proof of malicious intent.

2.4.5 Data Requirements

- **Input Data:** CERT logon, HTTP, file, device and email activity records.
- **Common Format:** Each event should contain an event ID, user/entity ID, timestamp, event type, source/target information and other useful event details.
- **Sequence Data:** Events will be arranged in time order and grouped into sessions or time windows.
- **Evaluation Labels:** CERT ground-truth malicious activity will be kept separately for testing and evaluation and will not be used as an input feature for the model.
- **Excluded Data:** Psychometric information will not be used as a model feature. The system will focus only on observable digital activity.

2.4.6 ML and Evaluation Requirements

- The system shall support an LSTM/GRU autoencoder as the baseline model.
- The system shall support a Transformer autoencoder.
- The system shall support a hybrid LSTM + Attention model.
- The primary evaluation metric shall be F1-score because the dataset contains many more normal records than malicious ones.
- Other evaluation measures shall include false-positive rate, training time, inference time and usefulness of explanations.
- Cold-start experiments shall compare transfer-learned models with models trained from scratch using a limited amount of behavioural history.
- The system shall keep a human-in-the-loop approach, meaning analysts will review the results before taking action. The system shall not automatically block users based only on their anomaly scores.

2.5 User Stories and Story Cards

The functional requirements above are additionally expressed as user stories to support iterative, analyst-centred development.

Story ID	US-01
User Story	As a Security Analyst, I want to view a ranked list of suspicious sessions, so that I can prioritise my triage effort on the highest-risk activity first.
Priority	High
Story Points	3
Acceptance Criteria	Sessions are ranked by anomaly score. The list can be filtered by date range and user/entity. Ranking updates when new sessions are scored.

Story ID	US-02
User Story	As a Security Analyst, I want to see the explanation behind an anomaly score, so that I can judge whether it reflects genuine risk before acting on it.
Priority	High
Story Points	5
Acceptance Criteria	Contributing events / sequence regions are highlighted for attention-based models. The explanation is shown alongside the session timeline. The system labels explanations as supporting evidence, not proof.

Story ID	US-03
User Story	As a Security Analyst, I want to confirm or dismiss an alert, so that my decision is recorded for audit and future model improvement.
Priority	High
Story Points	2

Acceptance Criteria	Confirm and Dismiss actions are available on every alert. Each decision is timestamped and attributed to the analyst. Dismissed alerts are retrievable from the audit log.
---------------------	---

Story ID	US-04
User Story	As a System Administrator, I want to manage analyst accounts and roles, so that access to sensitive behavioural data is properly controlled.
Priority	Medium
Story Points	3
Acceptance Criteria	Administrators can create, update and deactivate accounts. Roles restrict access to ingestion, review or admin functions. Role changes are captured in the audit log.

Story ID	US-05
User Story	As an ML Engineer, I want to fine-tune the generic model on a specific entity's limited history, so that new users are covered before enough entity-specific data has accumulated.
Priority	High
Story Points	8
Acceptance Criteria	Fine-tuning can run on day-5, day-10 and day-20 history subsets. Fine-tuned and from-scratch models can be compared on the same test split. Cold-start results are logged for later comparison.

Story ID	US-06
User Story	As a System Administrator, I want to review audit logs of analyst actions, so that all triage decisions remain traceable for compliance.
Priority	Medium
Story Points	3
Acceptance Criteria	Audit logs capture login, review and confirm/dismiss actions. Logs are filterable by analyst, date and action type. Logs cannot be edited or deleted by analysts.

Story ID	US-07
User Story	As a Security Analyst, I want the system to flag anomalies without automatically blocking users, so that a human always makes the final containment decision.
Priority	High
Story Points	2

Acceptance Criteria	No account or access is automatically revoked by the system. High-scoring sessions are surfaced for review, not enforcement. The distinction is documented for analysts and administrators.
---------------------	---

Story ID	US-08
User Story	As an ML Engineer, I want to compare F1-score, false-positive rate, training time and latency across the three architectures, so that I can select the best-performing model for deployment.
Priority	Medium
Story Points	5
Acceptance Criteria	A shared evaluation report is generated for all three model variants. Metrics are computed on the same held-out test split. Results can be exported for the comparison writeup.

3. DEVELOPMENT PROGRESS AND WEEKLY JOURNAL

The following journal records the major problems faced during project development and the decisions taken to solve them each week.

Week 1 — Dataset Selection and Problem Definition

Problem Faced:

During the initial phase of the project, we faced difficulty finding a dataset suitable for behavioural anomaly and insider-threat detection. Many commonly available cybersecurity datasets, such as HDFS, BGL and Thunderbird, mainly contain system or application logs and focus on system failures rather than user behaviour. We also realised that simply applying an autoencoder to a cybersecurity dataset would not make the project sufficiently different. Therefore, we needed to find both a suitable dataset and a clear research gap.

Solution:

- We explored and compared several publicly available cybersecurity and log datasets based on the type of events available, presence of malicious activity, ground-truth labels, dataset size and relevance to user behaviour.
- We selected the CERT Insider Threat Test Dataset as our primary dataset because it contains different types of user activity, including logon, file, HTTP, device and email events, along with known malicious insider scenarios.
- We reviewed existing approaches such as DeepLog, LogAnomaly, LogRobust and existing UEBA/SIEM solutions.

Outcome:

By the end of the week, we had selected the CERT dataset and refined the project scope and main ideas based on our literature review.

Week 2 — Handling and Preprocessing Large-Scale Data

Problem Faced:

After selecting CERT, we faced difficulties in understanding and preprocessing the dataset. The data was spread across multiple large files, with each file containing a different type of user activity and having its own structure. Combining these different events while maintaining their correct time order was challenging. Processing the complete dataset repeatedly also created memory and processing limitations. Since our models require sequential behavioural data rather than individual rows, the raw logs could not be directly used as model input.

Solution:

- We designed a common preprocessing pipeline to convert the different CERT event sources into a common event format.
- The pipeline includes selecting relevant features, converting different event types, standardising timestamps, sorting events chronologically and grouping events into meaningful user sessions.
- To process the dataset efficiently, we planned to avoid unnecessary columns and process large files in smaller chunks instead of loading everything into memory at once.
- We separated preprocessing from model training so that the raw data would not need to be processed repeatedly.
- The resulting pipeline converts:

Raw CERT Logs → Unified Events → Sessionization → Feature Encoding → Behavioural Sequences → Model Input

- We kept the ground-truth labels separate from the model input so that they would only be used during evaluation and would not affect model training.

Outcome:

We converted the raw and different CERT logs into a structured sequential format suitable for machine-learning experiments. This also provided a common input format for all three proposed models.

Week 3 — Model Architecture Selection and Experimental Design

Problem Faced:

Once the data pipeline was established, we faced the challenge of deciding which deep-learning architecture would be most suitable for detecting anomalies in behavioural sequences. LSTM/GRU models are suitable for sequential data, while Transformers may capture long-range relationships between events more effectively. We also considered combining both approaches. Instead of selecting a model only based on theory, we wanted to compare their performance through experiments.

Another challenge was selecting suitable evaluation measures because cybersecurity data contains many more normal activities than malicious ones. Using accuracy alone could therefore give misleading results.

Solution:

- We decided to compare three model variants using the same preprocessing pipeline:

- **LSTM/GRU Autoencoder** — baseline model for learning local and short-range behavioural patterns.
- **Transformer Autoencoder** — tests whether self-attention can better capture long-range relationships between events.
- **Hybrid LSTM + Attention** — combines sequential modelling with attention-based analysis.
- We established a common evaluation framework so that all three models could be tested under similar conditions.
- F1-score was selected as the primary metric, with false-positive rate, training time and inference latency as additional measures.
- We included transfer learning in the experimental design. A general model would first learn common behavioural patterns and then be adapted to individual users/entities using limited local data.
- We planned experiments using different amounts of available history, such as Day 5, Day 10 and Day 20, to check whether transfer learning could reduce the cold-start problem.
- For explainability, we planned to extract attention information from the attention-based models and show important events or parts of a behavioural sequence to the analyst instead of providing only an unexplained anomaly score.

Outcome:

We finalised the three-model architecture and evaluation strategy. The project therefore moved beyond simple anomaly detection and focused on model comparison, explainability and transfer learning as its main differentiating areas.

4. CURRENT PROJECT STATUS

Area	Status	Progress
Problem statement	Completed	Behavioural anomaly / insider-threat problem defined.
Dataset research	Completed	CERT selected as the primary evaluation dataset.
Literature review	In progress / refined	Existing approaches reviewed and novelty directions identified.
Project proposal	Completed	Proposal and scope documented.
Requirements	Completed	Functional and non-functional requirements drafted.
Use cases	Completed / documented	Major actors and system interactions identified.
Activity / Swimlane design	Completed / documented	Core workflow and component responsibilities mapped.
DFDs	Completed / documented	Level 0, Level 1 and Level 2 flows designed.
SRS	Completed / documented	IEEE-style requirements structure prepared.

Area	Status	Progress
User stories	Completed / documented	Story cards drafted for major analyst and admin workflows.
CERT preprocessing	In progress	Unified event schema and sessionization approach being developed.
Model implementation	Planned / next stage	LSTM/GRU, Transformer and hybrid implementations to be developed.
Transfer-learning experiments	Planned	Day 5 / 10 / 20 cold-start comparison planned.
Dashboard integration	Planned	Streamlit analyst workflow to be integrated with the ML service.

5. NEXT STEPS

- Finalize the CERT data ingestion and normalization implementation.
- Implement session-windowing/sessionization and generate model-ready behavioural sequences.
- Create a reproducible train/validation/test split that avoids temporal or entity leakage.
- Implement the LSTM/GRU autoencoder baseline in the Python ML service.
- Implement the Transformer autoencoder and hybrid LSTM + attention variants using the same input pipeline.
- Establish anomaly thresholds using training/validation data without using test-set labels.
- Run the architecture comparison using F1-score, false-positive rate, training time and inference latency.
- Run transfer-learning versus from-scratch experiments at different amounts of entity history.
- Implement attention extraction and analyst-facing explanation views.
- Integrate the Python ML service with the Spring Boot backend and Streamlit dashboard.
- Perform software testing, integration testing and final evaluation on held-out CERT data.

6. KNOWN LIMITATIONS

- Reconstruction error is not proof of maliciousness: an anomaly score indicates that behaviour is unfamiliar to the model and still requires human judgement.
- Transfer learning does not completely eliminate cold start: entity-specific behavioural quirks may still require local observations.
- Class imbalance: intrusion events are rare compared with normal activity, so precision, recall and F1 - score are more informative than raw accuracy.
- Synthetic evaluation data: CERT is a synthetic insider-threat dataset and therefore may not capture every characteristic of real enterprise environments.
- Explainability limitation: attention or highlighted events should be treated as supporting evidence rather than a guaranteed causal explanation of an alert.

7. SUMMARY

This progress report covers the project's selection, architecture, requirements, system analysis and ML experiment design. The project focuses on behavioural intrusion detection by analysing user activity sequences, with special attention to explainability, transfer learning and analyst review.

So far, the main challenges have been selecting a suitable dataset, identifying a clear research gap through literature review, handling large and different types of log data, and choosing ML models that can be compared fairly. These challenges helped us make practical decisions while keeping the main goal of the project unchanged.

In the next phase, we will focus on implementing the CERT preprocessing pipeline, building the three model variants, running controlled experiments and connecting the anomaly-detection service with the Spring Boot backend and Streamlit dashboard.